AgentDoctor v0.1.0 — Build Specification (TypeScript)

Version: 2.0 (Revised) **Date:** February 2026 **Author:** Anmol, MLDeep Systems **Target:** Claude Code **Goal:** Ship a production-ready, GitHub-publishable open-source TypeScript library and CLI for AI agent health diagnostics in ~19 focused days.

Table of Contents

1. Project Overview
2. Repository Structure
3. Core Library
4. The Six Pathologies — Detection Specifications
5. Log Parsers
6. Health Scoring Engine
7. Report Generator
8. CLI Tool
9. Fixture Generation & Test Data
10. Test Suite
11. CI/CD
12. Documentation
13. Packaging & Distribution
14. GitHub Repository Setup
15. Acceptance Criteria — Ship Checklist
16. v0.2.0 Backlog
17. Appendix A: Canonical Log Schemas

18. Appendix B: Health Score Formulas
19. Appendix C: Sample Report Output

---

1. Project Overview

1.1 What This Is

AgentDoctor is an open-source TypeScript library and CLI that analyzes AI agent execution logs, detects six known failure pathologies that cause agents to degrade in production, computes a composite health score, and generates a structured diagnostic report.

1.2 What This Is NOT

- **NOT** ML-based classification — rule-based pattern matching only
- **NOT** all 5 diagnostic layers — 3 of 5 (context health, tool reliability, instruction coherence). Recovery robustness and output quality baselines require longitudinal data
- **NOT** auto-remediation — text recommendations only
- **NOT** real-time monitoring — point-in-time analysis on uploaded logs
- **NOT** the proprietary ACT dataset — generic detection signatures only

1.3 Design Principles

1. **Zero external services.** No API keys, no cloud, no network calls. Runs locally on files.
2. **Framework-agnostic core.** Canonical internal format. Parsers translate framework-specific logs.
3. **Deterministic.** Same input → same output. No randomness, no LLM calls in analysis.
4. **Conservative on false positives.** Better to miss a marginal pathology than flag a false positive. <2% false positive rate target.
5. **Professional output.** Reports look like they came from a paid consultant.
6. **Strict TypeScript.** Full strict mode, no `any` in public API, exhaustive switch statements.

7. **Minimal dependencies.** 3 production deps. No heavy template engines, no ESM-only pitfalls.

1.4 Tech Stack

Concern	Choice	Why
Language	TypeScript 5.4+ strict	Self-documenting contracts, catches bugs before tests
Runtime	Node.js 20+ LTS	Stable, broad support
CLI	`commander`	Standard, lightweight, CJS-compatible
Config validation	`zod`	Type inference + runtime validation in one
Terminal colors	`picocolors`	14x smaller than chalk, no ESM-only issues
Testing	`vitest`	Fast, native TS support, compatible with Jest API
Linting	`eslint` + `@typescript-eslint`	Standard
Formatting	`prettier`	Standard
Build	`tsup` (ESM + CJS dual)	Simple, fast, handles shims
Package manager	`pnpm`	Fast, strict, disk-efficient
Report rendering	Template literals	No template engine dep needed for Markdown + JSON

Explicitly NOT using: `chalk` (ESM-only issues), `ora` (unnecessary for v0.1.0), `handlebars` (overkill for fixed report structure), `typedoc` (deferred to v0.2.0).

---

2. Repository Structure

```
agentdoctor/
├── .github/
```

```
│   ├── workflows/
│   │   ├── ci.yml                    # Lint + typecheck + test on every PR
│   │   └── release.yml               # Build + publish to npm on tag
│   ├── ISSUE_TEMPLATE/
│   │   ├── bug_report.md
│   │   ├── feature_request.md
│   │   └── pathology_report.md
│   ├── PULL_REQUEST_TEMPLATE.md
│   └── CODEOWNERS
├── docs/
│   ├── getting-started.md
│   ├── pathologies/
│   │   ├── README.md                 # Overview of all 7 (6 shipped, 1 coming)
│   │   ├── context-erosion.md
│   │   ├── tool-thrashing.md
│   │   ├── instruction-drift.md
│   │   ├── recovery-blindness.md
│   │   ├── hallucinated-tool-success.md
│   │   └── silent-degradation.md
│   ├── scoring.md
│   ├── parsers.md
│   ├── cli.md
│   └── configuration.md
├── src/
│   ├── index.ts                      # Public API exports
│   ├── models/
│   │   ├── index.ts
│   │   ├── canonical.ts              # Core data models
│   │   ├── findings.ts               # Pathology finding models
│   │   ├── scores.ts                 # Health score models
│   │   └── config.ts                 # Config schema (Zod)
│   ├── parsers/
│   │   ├── index.ts                  # Registry + auto-detection
│   │   ├── base.ts                   # Parser interface
│   │   ├── langchain.ts
│   │   ├── openai.ts
│   │   └── generic.ts
│   ├── detectors/
│   │   ├── index.ts                  # Detector registry
│   │   ├── base.ts                   # Detector interface
│   │   ├── context-erosion.ts
│   │   ├── tool-thrashing.ts
│   │   ├── instruction-drift.ts
│   │   ├── recovery-blindness.ts
│   │   ├── hallucinated-success.ts
│   │   └── silent-degradation.ts
│   ├── scoring/
```

```
│   │   ├── index.ts
│   │   ├── engine.ts
│   │   ├── context-health.ts
│   │   ├── tool-reliability.ts
│   │   └── instruction-coherence.ts
│   ├── reporting/
│   │   ├── index.ts
│   │   ├── engine.ts                # Report assembly orchestrator
│   │   ├── markdown.ts              # Markdown renderer (template literals)
│   │   └── json-report.ts           # JSON structured output
│   ├── cli/
│   │   ├── index.ts                 # Commander CLI entry point
│   │   ├── check.ts                 # `check` command
│   │   ├── score.ts                 # `score` command
│   │   └── formatters.ts            # Terminal output helpers
│   ├── pipeline.ts                  # End-to-end orchestrator
│   └── utils/
│       ├── tokens.ts
│       ├── similarity.ts
│       ├── statistics.ts
│       └── text.ts
├── tests/
│   ├── setup.ts
│   ├── helpers.ts                   # Test factories + shared utilities
│   ├── fixtures/
│   │   ├── logs/
│   │   │   ├── langchain/
│   │   │   │   ├── healthy-session.json
│   │   │   │   ├── context-erosion.json
│   │   │   │   ├── tool-thrashing.json
│   │   │   │   ├── instruction-drift.json
│   │   │   │   ├── recovery-blindness.json
│   │   │   │   ├── hallucinated-success.json
│   │   │   │   ├── silent-degradation.json
│   │   │   │   └── multi-pathology.json
│   │   │   ├── openai/
│   │   │   │   ├── healthy-session.jsonl
│   │   │   │   └── context-erosion.jsonl
│   │   │   └── generic/
│   │   │       ├── healthy-session.json
│   │   │       └── mixed-pathologies.json
│   │   └── configs/
│   │       ├── strict.json
│   │       └── relaxed.json
│   ├── unit/
│   │   ├── models.test.ts
│   │   ├── parsers/
```

```
│   │   │   ├── langchain.test.ts
│   │   │   ├── openai.test.ts
│   │   │   ├── generic.test.ts
│   │   │   └── detection.test.ts
│   │   ├── detectors/
│   │   │   ├── context-erosion.test.ts
│   │   │   ├── tool-thrashing.test.ts
│   │   │   ├── instruction-drift.test.ts
│   │   │   ├── recovery-blindness.test.ts
│   │   │   ├── hallucinated-success.test.ts
│   │   │   └── silent-degradation.test.ts
│   │   ├── scoring/
│   │   │   ├── context-health.test.ts
│   │   │   ├── tool-reliability.test.ts
│   │   │   ├── instruction-coherence.test.ts
│   │   │   └── engine.test.ts
│   │   ├── reporting/
│   │   │   ├── markdown.test.ts
│   │   │   └── json-report.test.ts
│   │   ├── config.test.ts
│   │   └── utils/
│   │       ├── tokens.test.ts
│   │       ├── similarity.test.ts
│   │       └── statistics.test.ts
│   ├── integration/
│   │   ├── pipeline.test.ts
│   │   └── cli.test.ts
│   └── regression/
│       ├── false-positives.test.ts
│       └── detection-stability.test.ts
├── scripts/
│   └── generate-fixtures.ts          # Programmatic fixture generation
├── examples/
│   ├── basic-usage.ts
│   ├── custom-thresholds.ts
│   └── sample-logs/
│       ├── README.md
│       ├── healthy-agent.json
│       └── degraded-agent.json
├── package.json
├── tsconfig.json
├── tsup.config.ts
├── vitest.config.ts
├── .eslintrc.cjs
├── .prettierrc
├── .gitignore
├── README.md
```

```
├── LICENSE                          # Apache 2.0
├── CHANGELOG.md
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
└── SECURITY.md
```

What's not here (deferred to v0.2.0): `typedoc.json`, docs CI workflow, CrewAI parser, Vercel AI parser, HTML reporter, `templates/` directory, `config` CLI command files.

---

3. Core Library

3.1 Canonical Data Model (`models/canonical.ts`)

All parsers convert into this format. All detectors operate on it.

```
export enum Role {
  System = "system",
  User = "user",
  Assistant = "assistant",
  Tool = "tool",
}

export enum ToolCallStatus {
  Success = "success",
  Error = "error",
  Timeout = "timeout",
  Partial = "partial",
  Unknown = "unknown",
}

export interface Message {
  readonly role: Role;
  readonly content: string;
  readonly timestamp?: string;
  readonly tokenCount?: number;
  readonly metadata?: Record<string, unknown>;
}

export interface ToolCall {
  readonly toolName: string;
  readonly toolInput: Record<string, unknown>;
  readonly toolOutput?: string;
  readonly toolOutputRaw?: unknown;
```

```
  readonly status: ToolCallStatus;
  readonly latencyMs?: number;
  readonly timestamp?: string;
  readonly errorMessage?: string;
  readonly retryCount: number;
  readonly metadata?: Record<string, unknown>;
}

export interface Turn {
  readonly messages: readonly Message[];
  readonly toolCalls: readonly ToolCall[];
  readonly turnIndex: number;
  readonly contextTokenCount?: number;
  readonly timestamp?: string;
  readonly metadata?: Record<string, unknown>;
}

export interface ToolSchema {
  readonly name: string;
  readonly description?: string;
  readonly parameters?: Record<string, unknown>;
}

export interface AgentSession {
  readonly sessionId: string;
  readonly turns: readonly Turn[];
  readonly systemPrompt?: string;
  readonly toolSchemas: readonly ToolSchema[];
  readonly framework?: string;
  readonly agentName?: string;
  readonly startTime?: string;
  readonly endTime?: string;
  readonly metadata?: Record<string, unknown>;
}

export interface AgentLogBundle {
  readonly sessions: readonly AgentSession[];
  readonly sourceFiles: readonly string[];
  readonly frameworkDetected?: string;
}

// Computed helpers (pure functions, no methods on interfaces)
export function totalTurns(session: AgentSession): number {
  return session.turns.length;
}

export function totalToolCalls(session: AgentSession): number {
```

```
  return session.turns.reduce((sum, t) => sum + t.toolCalls.length, 0);
}

export function totalMessages(session: AgentSession): number {
  return session.turns.reduce((sum, t) => sum + t.messages.length, 0);
}
```

3.2 Findings Model (`models/findings.ts`)

```
export enum Pathology {
  ContextErosion = "context_erosion",
  ToolThrashing = "tool_thrashing",
  InstructionDrift = "instruction_drift",
  RecoveryBlindness = "recovery_blindness",
  HallucinatedToolSuccess = "hallucinated_tool_success",
  SilentDegradation = "silent_degradation",
}

export enum Severity {
  Critical = "critical",
  Warning = "warning",
  Info = "info",
}

export interface Evidence {
  readonly description: string;
  readonly turnIndex?: number;
  readonly toolCallIndex?: number;
  readonly sessionId?: string;
  readonly rawData?: Record<string, unknown>;
}

export interface Finding {
  readonly pathology: Pathology;
  readonly severity: Severity;
  readonly title: string;
  readonly description: string;
  readonly evidence: readonly Evidence[];
  readonly recommendation: string;
  readonly affectedSessions: readonly string[];
  readonly confidence: number;    // 0.0-1.0
  readonly metadata?: Record<string, unknown>;
}

export interface DiagnosticResult {
  readonly findings: readonly Finding[];
```

```
  readonly sessionsAnalyzed: number;
  readonly turnsAnalyzed: number;
  readonly toolCallsAnalyzed: number;
  readonly analysisTimestamp: string;
  readonly configUsed: Record<string, unknown>;
}

export function criticalCount(result: DiagnosticResult): number {
  return result.findings.filter((f) => f.severity === Severity.Critical).length;
}

export function warningCount(result: DiagnosticResult): number {
  return result.findings.filter((f) => f.severity === Severity.Warning).length;
}

export function pathologiesDetected(result: DiagnosticResult): Set<Pathology> {
  return new Set(result.findings.map((f) => f.pathology));
}
```

3.3 Score Model (`models/scores.ts`)

```
export type Grade = "A" | "B" | "C" | "D" | "F";

export interface LayerScore {
  readonly name: string;
  readonly score: number;        // 0-100
  readonly grade: Grade;
  readonly summary: string;
  readonly components: Record<string, number>;
  readonly flags: readonly string[];
}

export interface HealthScore {
  readonly overallScore: number;
  readonly overallGrade: Grade;
  readonly layers: readonly LayerScore[];
  readonly assessedLayers: number;
  readonly unassessedLayers: readonly string[];
  readonly summary: string;
}

export function gradeFromScore(score: number): Grade {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  if (score >= 60) return "D";
```

```
  return "F";
}
```

3.4 Config Schema (`models/config.ts`)

```
import { z } from "zod";

const threshold01 = z.number().min(0).max(1);
const thresholdPositive = z.number().positive();

export const AgentDoctorConfigSchema = z.object({
  contextErosion: z.object({
    growthRateWarning: thresholdPositive.default(500),
    growthRateCritical: thresholdPositive.default(2000),
    monotonicThreshold: threshold01.default(0.8),
    windowPctCritical: threshold01.default(0.8),
    assumedWindowSize: thresholdPositive.default(128000),
    instructionShareCritical: threshold01.default(0.02),
    instructionShareWarning: threshold01.default(0.05),
    instructionShareStartMin: threshold01.default(0.15),
    staleContentThreshold: threshold01.default(0.6),
    staleTurnLookback: thresholdPositive.int().default(10),
  }).default({}),

  toolThrashing: z.object({
    windowSize: thresholdPositive.int().default(5),
    repetitionWarning: thresholdPositive.int().default(3),
    repetitionCritical: thresholdPositive.int().default(5),
    inputSimilarityThreshold: threshold01.default(0.7),
    oscillationMinCycles: thresholdPositive.int().default(3),
    oscillationCriticalCycles: thresholdPositive.int().default(5),
    callsPerTurnWarning: thresholdPositive.int().default(8),
  }).default({}),

  instructionDrift: z.object({
    checkToolReferences: z.boolean().default(true),
    checkContradictions: z.boolean().default(true),
  }).default({}),

  recoveryBlindness: z.object({
    maxBlindRetries: thresholdPositive.int().default(3),
    errorRateWarning: threshold01.default(0.2),
    errorRateCritical: threshold01.default(0.5),
    flagUntested: z.boolean().default(true),
  }).default({}),
```

```
  hallucinatedSuccess: z.object({
    errorAcknowledgmentKeywords: z.array(z.string()).default([
      "failed", "error", "couldn't", "unable", "issue",
      "problem", "sorry", "unfortunately",
    ]),
  }).default({}),

  silentDegradation: z.object({
    withinSessionDropThreshold: threshold01.default(0.2),
  }).default({}),

  scoring: z.object({
    criticalPenalty: thresholdPositive.default(5),
    criticalPenaltyMax: thresholdPositive.default(25),
    warningPenalty: thresholdPositive.default(2),
    warningPenaltyMax: thresholdPositive.default(10),
  }).default({}),
}).default({});

export type AgentDoctorConfig = z.infer<typeof AgentDoctorConfigSchema>;

export function loadConfig(overrides?: Record<string, unknown>): AgentDoctorConfi
  return AgentDoctorConfigSchema.parse(overrides ?? {});
}

export function getDefaultConfig(): AgentDoctorConfig {
  return AgentDoctorConfigSchema.parse({});
}
```

3.5 Pipeline Orchestrator (`pipeline.ts`)

```
export type OutputFormat = "markdown" | "json";

export interface AnalyzeOptions {
  logFiles: string[];
  config?: Partial<AgentDoctorConfig>;
  outputFormat?: OutputFormat;       // default: "markdown"
  outputPath?: string;               // write to file, or return as string
  pathologyFilter?: Pathology[];
  severityThreshold?: Severity;      // default: Warning (filter out Info-only)
  stdin?: boolean;                   // read from stdin instead of files
}

export interface AnalyzeResult {
  diagnostics: DiagnosticResult;
  healthScore: HealthScore;
```

```
  report: string;
}

/**
 * Run the full diagnostic pipeline.
 *
 * 1. Read log files (or stdin)
 * 2. Auto-detect framework, parse to canonical format
 * 3. Run detectors (all, or filtered subset)
 * 4. Compute health scores from findings + bundle
 * 5. Generate report (markdown or json)
 * 6. Write output if outputPath specified
 *
 * @returns AnalyzeResult with diagnostics, scores, and rendered report
 */
export async function analyze(options: AnalyzeOptions): Promise<AnalyzeResult>;
```

3.6 Public API Exports (`index.ts`)

```
// Core pipeline
export { analyze } from "./pipeline";
export type { AnalyzeOptions, AnalyzeResult, OutputFormat } from "./pipeline";

// Models
export { Role, ToolCallStatus } from "./models/canonical";
export type { Message, ToolCall, Turn, AgentSession, ToolSchema, AgentLogBundle }
export { Pathology, Severity } from "./models/findings";
export type { Finding, Evidence, DiagnosticResult } from "./models/findings";
export type { HealthScore, LayerScore, Grade } from "./models/scores";
export { gradeFromScore } from "./models/scores";

// Parsers (for advanced usage / extension)
export { detectAndParse } from "./parsers";
export type { BaseParser } from "./parsers/base";

// Detectors (for advanced usage / extension)
export { getAllDetectors } from "./detectors";
export type { BaseDetector } from "./detectors/base";

// Scoring
export { computeHealthScore } from "./scoring/engine";

// Config
export { getDefaultConfig, loadConfig, AgentDoctorConfigSchema } from "./models/c
export type { AgentDoctorConfig } from "./models/config";
```

4. The Six Pathologies — Detection Specifications

v0.1.0 ships 6 detectors. Goal Hijacking is deferred to v0.2.0 (requires ACT dataset calibration for acceptable precision).

4.1 Base Detector Interface (`detectors/base.ts`)

```
export interface BaseDetector {
  readonly pathology: Pathology;
  readonly name: string;
  detect(bundle: AgentLogBundle, config: AgentDoctorConfig): Finding[];
}
```

4.2 Detector Registry (`detectors/index.ts`)

```
export function getAllDetectors(): BaseDetector[] {
  return [
    new ContextErosionDetector(),
    new ToolThrashingDetector(),
    new InstructionDriftDetector(),
    new RecoveryBlindnessDetector(),
    new HallucinatedSuccessDetector(),
    new SilentDegradationDetector(),
  ];
}

export function getDetectorsByPathology(pathologies: Pathology[]): BaseDetector[]
  return getAllDetectors().filter((d) => pathologies.includes(d.pathology));
}
```

4.3 Context Erosion (`detectors/context-erosion.ts`)

Detects: Context window growing unchecked, instruction dilution, declining signal-to-noise ratio.

Rules:

1. **Monotonic growth.** `contextTokenCount` increases in ≥80% of consecutive turn pairs. No plateau or reduction observed.
    - `Warning` if avg growth > 500 tokens/turn. `Critical` if > 2,000 tokens/turn OR final context > 80% of assumed window (128K).

- Config: `contextErosion.growthRateWarning`, `growthRateCritical`, `monotonicThreshold`, `windowPctCritical`, `assumedWindowSize`

2. **Instruction share decline.** `systemPromptTokens / contextTokenCount` drops below 5% when it started above 15%.
   - `Critical` if < 2%. `Warning` if < 5%.
   - Config: `contextErosion.instructionShareCritical`, `instructionShareWarning`, `instructionShareStartMin`
3. **Stale content.** >60% of context tokens from turns >10 ago, no summarization evident (token count never decreases).
   - `Warning`.
   - Config: `contextErosion.staleContentThreshold`, `staleTurnLookback`

Minimum data: 5+ turns with `contextTokenCount`. If missing, return empty findings + metadata warning.

Evidence: Turn indices, start/end token counts, instruction share start/end, token count series.

Recommendation: `"Implement context management: summarize tool outputs older than N turns, add TTL-based memory pruning, or implement a sliding window with instruction anchoring."`

4.4 Tool Thrashing (`detectors/tool-thrashing.ts`)

Detects: Repeated same-tool calls with similar inputs, oscillating tool sequences, excessive calls per turn.

Rules:

1. **Repetitive same-tool.** Sliding window of 5 consecutive tool calls. Same tool ≥3 times AND Jaccard similarity of flattened inputs > 0.7.
   - `Critical` if ≥5 reps. `Warning` if 3–4.
   - Config: `toolThrashing.windowSize`, `repetitionWarning`, `repetitionCritical`, `inputSimilarityThreshold`
   - Similarity: `flattenDict()` → dot-notation keys → Jaccard index of key-value pair sets.
2. **Tool oscillation.** Detect A→B→A→B or A→B→C→A→B→C patterns repeating ≥3

consecutive cycles.

- `Warning` at 3. `Critical` at ≥5.
- Config: `toolThrashing.oscillationMinCycles`, `oscillationCriticalCycles`

3. **High calls-per-turn.** Single turn with >8 tool calls.
   - `Warning`.
   - Config: `toolThrashing.callsPerTurnWarning`

Evidence: Tool calls involved (inputs truncated to 200 chars), similarity scores, pattern description.

Recommendation: `"Review tool descriptions for ambiguity. Add 'use when' and 'do not use when' clauses. Consider a tool-call budget per task."`

4.5 Instruction Drift (`detectors/instruction-drift.ts`)

Detects: Contradictions between system prompt, tool schemas, and injected instructions.

Rules:

1. **Tool reference mismatch.** Extract tool names from system prompt (backtick-wrapped, "use the X tool", "call X"). Compare against `toolSchemas[].name`.
   - `Critical` for phantom tools (in prompt, not in schemas). `Warning` for orphaned tools (in schemas, never referenced).
   - Config: `instructionDrift.checkToolReferences`
2. **Contradictory directives.** Scan for phrases with "always", "never", "must", "do not". Flag opposing directives targeting same subject.
   - `Info` (heuristic, flagged for human review).
   - Config: `instructionDrift.checkContradictions`
3. **Missing system prompt.** No system prompt found.
   - `Info`.

Evidence: Mismatch details (tool name → present in prompt / present in schema), directive text pairs.

Recommendation: `"Create a unified instruction inventory. Version-control all instruction surfaces together. Run behavioral regression tests after changes."`

4.6 Recovery Blindness (`detectors/recovery-blindness.ts`)

Detects: Tool failures with no fallback — no retry, no alternative, no graceful degradation.

Rules:

1. **Unhandled failures.** For each `ToolCall` with status Error/Timeout, examine next agent action:
   - Retry (acceptable unless retry also fails)
   - Different tool (acceptable — fallback)
   - Acknowledgment message with keywords: "failed", "error", "couldn't", "unable", "issue", "problem", "unfortunately" (acceptable — graceful degradation)
   - Proceeds as if succeeded → `Critical`
   - Retries identical inputs ≥3 times → `Warning`
   - Config: `recoveryBlindness.maxBlindRetries`
2. **Per-tool error rate.** Tool with >20% errors and no recovery patterns.
   - `Critical` if >50%. `Warning` if >20%.
   - Config: `recoveryBlindness.errorRateWarning` , `errorRateCritical`
3. **Untested failure handling.** Zero tool errors AND zero evidence of retry/fallback patterns.
   - `Info` .
   - Config: `recoveryBlindness.flagUntested`

Evidence: Failed calls, next action taken, per-tool error rates.

Recommendation: `"Define recovery policies per tool: retry with backoff for transient errors, fallback for capability errors, escalate for persistent failures."`

4.7 Hallucinated Tool Success (`detectors/hallucinated-success.ts`)

Detects: Agent claiming tool succeeded when it failed or returned unexpected results.

Rules:

1. **Status/output mismatch.** Tool status == Error but assistant's next message

contains no acknowledgment keywords and no retry language. Agent proceeds as if succeeded.

- `Critical`.
- Config: `hallucinatedSuccess.errorAcknowledgmentKeywords`

2. **Empty output treated as success.** Tool returns empty/null/error-containing JSON but assistant makes specific data claims (numbers, URLs, proper nouns) not in prior context.
   - `Warning` (heuristic).
3. **Partial result acceptance.** Tool output contains truncation indicators ("truncated", "partial", "limit exceeded", "more results available"). Agent treats as complete.
   - `Warning`.

Evidence: Tool call, actual output, assistant's subsequent message.

Recommendation: `"Add output validation after every tool call. Verify response schema, check for error indicators, validate critical fields before proceeding."`

4.8 Silent Degradation (`detectors/silent-degradation.ts`)

Detects: Gradual performance decline within sessions.

v0.1.0 ships rules 2 and 3 only. Cross-session analysis (rule 1) deferred to v0.2.0 — requires multi-session data most users won't have yet.

Rules:

1. ~~Cross-session quality proxy decline~~ → **Deferred to v0.2.0**
2. **Within-session performance drop.** Divide session into thirds by turn count. Compare quality proxies (tool success rate, avg tool calls per turn, error rate) first third vs. last third. Flag if >20% decline.
   - `Warning`.
   - Config: `silentDegradation.withinSessionDropThreshold`
3. **Compounding pathology indicator.** If ≥3 other pathologies detected in the same bundle, flag as degradation signal.
   - `Info`.

Evidence: Quality proxy comparison (first third vs. last third), other detected

pathologies.

Recommendation: "Establish performance baselines and measure weekly. Use distribution-based alerting (current vs. rolling 4-week average) rather than threshold alerts."

---

5. Log Parsers

5.1 Parser Interface (`parsers/base.ts`)

```
export interface BaseParser {
  /** Check if this parser can handle the file. `sample` is first 2KB. */
  canParse(filePath: string, sample: string): boolean;
  /** Parse file content into AgentSession objects. */
  parse(filePath: string, content: string): AgentSession[];
  /** Framework name for display/logging. */
  readonly frameworkName: string;
}
```

5.2 Registry & Auto-Detection (`parsers/index.ts`)

```
// Most specific first, generic last
const PARSER_REGISTRY: BaseParser[] = [
  new LangChainParser(),
  new OpenAIParser(),
  new GenericParser(),
];

export function detectAndParse(filePath: string, content: string): AgentSession[]
  const sample = content.slice(0, 2048);
  for (const parser of PARSER_REGISTRY) {
    if (parser.canParse(filePath, sample)) {
      return parser.parse(filePath, content);
    }
  }
  const tried = PARSER_REGISTRY.map((p) => p.frameworkName).join(", ");
  throw new Error(
    `Could not parse ${filePath}. Tried: ${tried}. ` +
    `See docs/parsers.md for supported formats, or use the generic JSON format.`
  );
}
```

```
export function detectFramework(filePath: string, content: string): string | null
  const sample = content.slice(0, 2048);
  for (const parser of PARSER_REGISTRY) {
    if (parser.canParse(filePath, sample)) return parser.frameworkName;
  }
  return null;
}
```

5.3 LangChain Parser (`parsers/langchain.ts`)

`canParse` : JSON with `"run_id"` , `"parent_run_id"` , `"serialized"` , or `"type"` values `"llm"` / `"chain"` / `"tool"` . Also LangChain.js callback patterns with `"name"` , `"tags"` , `"inputs"` , `"outputs"` .

Parsing:

- Parse callback handler JSON (Python + JS formats)
- Each LLM call + associated tool calls = one Turn
- Extract system prompt from first chain input or first system message
- Extract tool schemas from tool definitions if present
- Split multi-run_id files into separate sessions
- Token counts from `usage` metadata

5.4 OpenAI Parser (`parsers/openai.ts`)

`canParse` : JSONL with `"model"` , `"messages"` , `"choices"` , `"usage"` . Or JSON array of request/response pairs.

Parsing:

- Each request/response pair = one Turn
- Tool calls from `choices[0].message.tool_calls`
- Map `function.name` → toolName, `function.arguments` → toolInput (JSON-parsed)
- Token counts from `usage`
- System prompt from first request's system message
- Handle both Chat Completions and Assistants API formats

5.5 Generic Parser (`parsers/generic.ts`)

`canParse` : Fallback. Requires JSON with `"messages"` , `"turns"` , or `"events"` array.

Parsing:

- Flexible JSON parser for common patterns
- Best-effort extraction, populate what's available
- Log warnings for missing fields
- This is the parser that catches custom/proprietary formats

5.6 Utilities (`parsers/utils.ts`)

Not in scope: CrewAI parser, Vercel AI SDK parser. See CONTRIBUTING.md for how to add parsers.

```
/** ~4 chars per token. Intentionally simple — no tiktoken dependency. */
export function estimateTokenCount(text: string): number {
  return Math.max(1, Math.ceil(text.length / 4));
}

export async function readFileContent(filePath: string): Promise<string>;
export function safeJsonParse(content: string): unknown[];
export function extractTimestamp(
  data: Record<string, unknown>,
  candidates?: string[]
): string | undefined;
```

---

6. Health Scoring Engine

6.1 Three Layers

Layer 1: Context Health (`scoring/context-health.ts`)

Component	Weight	100 (Healthy)	0 (Critical)
Growth management	0.35	Context plateaus or is pruned	Monotonic growth
Instruction share	0.35	>15% of context	<2% of context
Stale content	0.30	<20% stale	>80% stale

Returns `null` if no token counts available.

Layer 2: Tool Reliability (`scoring/tool-reliability.ts`)

Component	Weight	100 (Healthy)	0 (Critical)
Success rate	0.40	>95%	<50%
Calls-per-turn	0.30	≤2 avg	>8 avg
Thrashing score	0.30	0 episodes	>3 episodes

Layer 3: Instruction Coherence (`scoring/instruction-coherence.ts`)

Component	Weight	100 (Healthy)	0 (Critical)
Prompt-schema alignment	0.50	All correct	Multiple phantom/orphan
Instruction consistency	0.30	No contradictions	Multiple found
System prompt present	0.20	Yes	No

6.2 Composite Score (`scoring/engine.ts`)

```
const LAYER_WEIGHTS: Record<string, number> = {
  context_health: 0.40,
  tool_reliability: 0.35,
  instruction_coherence: 0.25,
};

/**
 * 1. Compute each layer score (or null if insufficient data)
 * 2. For null layers, redistribute weight proportionally
 * 3. Apply finding penalties:
 *    - Each CRITICAL: -5 pts (max -25)
 *    - Each WARNING: -2 pts (max -10)
 * 4. Clamp to [0, 100]
 */
export function computeHealthScore(
  bundle: AgentLogBundle,
  diagnostics: DiagnosticResult,
  config: AgentDoctorConfig,
): HealthScore;
```

7. Report Generator

7.1 Two Formats: Markdown + JSON

No HTML renderer in v0.1.0. Markdown covers human-readable. JSON covers machine-readable / piping to other tools.

7.2 Report Structure (Both Formats)

```
1. EXECUTIVE SUMMARY
   - Overall score + grade
   - Pathology count by severity
   - 2-3 sentence summary
   - Assessed vs. unassessed layers

2. HEALTH SCORE BREAKDOWN
   - Per-layer scores with grade + components
   - Unassessed layers + why

3. FINDINGS (sorted: critical → warning → info)
   Per finding: pathology, severity badge, title, description,
   evidence (collapsible <details> in Markdown), recommendation, confidence

4. WHAT THIS ANALYSIS COULD NOT ASSESS
   - Recovery robustness (requires simulated failures)
   - Output quality baselines (requires longitudinal data)
   - Goal hijacking (coming in v0.2.0)
   - "For continuous monitoring with all 5 diagnostic layers,
     auto-remediation, and real-time alerting, visit mldeep.systems/agentdoctor"

5. RECOMMENDATIONS (ranked by impact)
   - Priority-ordered, with expected impact + complexity (low/med/high)

6. APPENDIX
   - Files, sessions, turns, tool calls analyzed
   - Configuration (all thresholds used)
   - "Generated by AgentDoctor v0.1.0 (agentdoctor.dev) — by MLDeep Systems"
```

7.3 Markdown Renderer (`reporting/markdown.ts`)

Built with template literal functions. No template engine dependency.

- Severity badges: 🔴 CRITICAL, 🟡 WARNING, ℹ️ INFO

- Score bars: ▓▓▓

`--config`	string	defaults	Path to JSON config file
`--pathology`	string	all	Run single detector only
`--severity-threshold`	`warning | critical`	`warning`	Minimum severity to report/fail on
`--stdin`	boolean	false	Read log from stdin
`-q, --quiet`	boolean	false	Exit code only, no output
`-v, --verbose`	boolean	false	Debug logging
`--json`	boolean	false	Shorthand for `--format json`

8.3 Terminal Output

- Score colored via `picocolors` : green (A/B), yellow (C), red (D/F)
- Critical findings in red, warnings in yellow
- After score display: `"Full diagnostic by MLDeep Systems → mldeep.systems/agentdoctor"`
- Error messages on parse failure include which parsers were tried and why they failed (eliminates need for separate `validate` command)

8.4 Exit Codes

Code	Meaning
0	Healthy — no findings at or above severity threshold
1	Warnings detected (or critical, if threshold is `warning`)
2	Critical findings detected
3	Analysis error — parse failure or runtime error

8.5 Stdin Support

When `--stdin` is passed, read entire stdin as a string, auto-detect format, parse, run pipeline. Enables:

```
cat agent.log | npx agentdoctor check --stdin
curl https://my-api/logs | npx agentdoctor check --stdin --json
```

---

9. Fixture Generation & Test Data

9.1 Fixture Generator Script (`scripts/generate-fixtures.ts`)

Instead of hand-authoring 12+ JSON files, build fixtures programmatically:

```
/**
 * Generates all test fixture files from canonical data.
 *
 * 1. Define sessions in canonical format (AgentSession) using builder functions
 * 2. Serialize to each framework format (LangChain, OpenAI, Generic)
 * 3. Write to tests/fixtures/logs/
 *
 * Benefits:
 * - Pathology characteristics are precisely controlled
 * - Adding a new parser format means writing one serializer
 * - Fixtures are always consistent with each other
 * - Threshold-exceeding values are computed, not guessed
 */

// Builder functions for common patterns:
function buildHealthySession(turns: number): AgentSession;
function buildContextErosionSession(opts: { turns: number; startTokens: number; e
function buildToolThrashingSession(opts: { repetitions: number; oscillationCycles
// ... one per pathology

// Serializers for each format:
function toLangChainFormat(session: AgentSession): object;
function toOpenAIFormat(session: AgentSession): string;  // JSONL
function toGenericFormat(session: AgentSession): object;

// Main: generate all fixtures
async function main(): Promise<void>;
```

9.2 Fixture Sizes (Reduced)

Fixture	Turns	Sessions	Rationale
healthy-session	15	1	Enough to show stability across all metrics

context-erosion	20	1	Growth from 3K→50K tokens in 20 turns
tool-thrashing	15	1	3 distinct episodes fit in 15 turns
instruction-drift	10	1	Only needs system prompt + schema mismatch
recovery-blindness	12	1	4 failure scenarios × ~3 turns each
hallucinated-success	10	1	2–3 hallucination instances
silent-degradation	10	3	3 sessions × 10 turns for within-session drop
multi-pathology	25	1	Context erosion + tool thrashing + recovery blindness

9.3 Required Fixture Files

LangChain (8): healthy + 6 pathologies + multi-pathology **OpenAI (2):** healthy + context-erosion **Generic (2):** healthy + mixed-pathologies

Total: 12 fixture files. All generated by `scripts/generate-fixtures.ts`.

---

10. Test Suite

10.1 Philosophy

- Every detector has positive AND negative (false-positive) tests
- No golden file tests — use structural assertions (right pathology, right severity, evidence non-empty)
- Fixtures are generated artifacts, not hand-authored
- Tests are deterministic: no randomness, no network, no timing

10.2 Unit Tests

```
models.test.ts
```

```
describe("gradeFromScore", () => {
  it("returns correct grade for each boundary", () => {});
  it("handles 0 and 100", () => {});
});
describe("criticalCount / warningCount", () => {
  it("counts mixed severities correctly", () => {});
  it("returns 0 for empty findings", () => {});
});
describe("pathologiesDetected", () => {
  it("returns unique set", () => {});
});
describe("totalTurns / totalToolCalls / totalMessages", () => {
  it("computes correctly", () => {});
});
```

parsers/langchain.test.ts

```
describe("LangChainParser", () => {
  it("identifies valid LangChain logs via canParse", () => {});
  it("rejects non-LangChain logs", () => {});
  it("parses healthy session with correct turn count", () => {});
  it("extracts system prompt", () => {});
  it("extracts tool calls with name, input, output, status", () => {});
  it("extracts token counts from usage metadata", () => {});
  it("handles missing optional fields gracefully", () => {});
  it("throws on empty content", () => {});
  it("throws on malformed JSON", () => {});
});
```

(Similar for `openai.test.ts`, `generic.test.ts`)

parsers/detection.test.ts

```
describe("Parser Auto-Detection", () => {
  it("selects LangChain for LangChain logs", () => {});
  it("selects OpenAI for OpenAI JSONL", () => {});
  it("falls back to generic", () => {});
  it("throws for unsupported format with helpful message", () => {});
});
```

detectors/context-erosion.test.ts

```
describe("ContextErosionDetector", () => {
  it("detects monotonic growth", () => {});
  it("detects instruction share dilution", () => {});
  it("returns no findings on healthy logs", () => {});
  it("does not flag growth that plateaus", () => {});
  it("returns Critical for extreme growth rate", () => {});
  it("returns Warning for moderate growth rate", () => {});
  it("returns empty when no token counts available", () => {});
  it("respects custom thresholds from config", () => {});
  it("includes evidence with turn indices and token counts", () => {});
  it("includes recommendation text", () => {});
  it("requires minimum 5 turns with token counts", () => {});
});
```

(Similar ~10 test cases for EACH of the remaining 5 detectors. Each has: positive detection, negative/false-positive, severity levels, config override, evidence validation.)

`scoring/engine.test.ts`

```
describe("HealthScoreEngine", () => {
  it("healthy session scores above 80", () => {});
  it("degraded session scores below 60", () => {});
  it("applies critical penalty (-5 per, max -25)", () => {});
  it("applies warning penalty (-2 per, max -10)", () => {});
  it("clamps score to 0 minimum", () => {});
  it("redistributes weight when layer is null", () => {});
  it("is deterministic", () => {});
  it("includes 3 assessed + 2 unassessed layers", () => {});
  it("generates non-empty summary", () => {});
});
```

`reporting/markdown.test.ts`

```
describe("MarkdownReporter", () => {
  it("contains all 6 sections", () => {});
  it("displays score and grade", () => {});
  it("sorts findings by severity", () => {});
  it("includes evidence blocks", () => {});
  it("includes unassessed layers section", () => {});
  it("includes MLDeep Systems footer", () => {});
  it("produces clean report with zero findings", () => {});
});
```

reporting/json-report.test.ts

```
describe("JsonReporter", () => {
  it("output is valid JSON", () => {});
  it("contains diagnostics, healthScore, and metadata", () => {});
  it("findings match DiagnosticResult structure", () => {});
});
```

config.test.ts

```
describe("Config", () => {
  it("loads defaults without error", () => {});
  it("custom overrides work", () => {});
  it("unspecified values fall back to defaults", () => {});
  it("rejects invalid thresholds (negative, out of range)", () => {});
  it("loads from JSON file content", () => {});
});
```

utils/*.test.ts

```
// tokens.test.ts
describe("estimateTokenCount", () => {
  it("estimates ~4 chars per token", () => {});
  it("returns 1 for empty string", () => {});
});

// similarity.test.ts
describe("Similarity", () => {
  it("flattenDict handles nested objects", () => {});
  it("jaccardSimilarity returns 1.0 for identical sets", () => {});
  it("jaccardSimilarity returns 0.0 for disjoint sets", () => {});
  it("handles partial overlap correctly", () => {});
});

// statistics.test.ts
describe("Statistics", () => {
  it("isMonotonic detects increasing sequences", () => {});
  it("isMonotonic respects tolerance threshold", () => {});
  it("trendDirection classifies correctly", () => {});
});
```

10.3 Integration Tests

pipeline.test.ts

```
describe("Pipeline", () => {
  it("healthy log → score >80, zero critical, valid markdown", () => {});
  it("context erosion → pathology detected, score <70", () => {});
  it("tool thrashing → pathology detected, recommendations present", () => {});
  it("multi-pathology → all expected pathologies detected", () => {});
  it("multiple input files handled correctly", () => {});
  it("both output formats produce valid output", () => {});
  it("pathologyFilter limits detectors", () => {});
  it("custom config propagates to detectors", () => {});
  it("severityThreshold filters findings in output", () => {});
});
```

cli.test.ts

```
describe("CLI", () => {
  it("'check file.json' runs without error", () => {});
  it("'--output' writes report to file", () => {});
  it("'--format json' produces JSON", () => {});
  it("'--config' loads custom thresholds", () => {});
  it("'--pathology' runs single detector", () => {});
  it("'--severity-threshold critical' filters warnings", () => {});
  it("'score' command outputs score and grade", () => {});
  it("exit code 0 on healthy", () => {});
  it("exit code 1 on warnings", () => {});
  it("exit code 2 on critical", () => {});
  it("'-q' suppresses output", () => {});
  it("'--version' prints version", () => {});
  it("parse failure includes which parsers were tried", () => {});
});
```

10.4 Regression Tests

false-positives.test.ts

```
describe("False Positive Regression", () => {
  const healthyFixtures = [
    "langchain/healthy-session.json",
    "openai/healthy-session.jsonl",
    "generic/healthy-session.json",
  ];
```

```
  it.each(healthyFixtures)("no critical findings on %s", (fixture) => {});
  it.each(healthyFixtures)("no warning findings on %s", (fixture) => {});
  it.each(healthyFixtures)("health score >= 80 on %s", (fixture) => {});
});
```

detection-stability.test.ts

```
describe("Detection Stability", () => {
  it("context erosion consistently detected", () => {});
  it("tool thrashing consistently detected", () => {});
  it("instruction drift consistently detected", () => {});
  it("recovery blindness consistently detected", () => {});
  it("hallucinated success consistently detected", () => {});
  it("silent degradation consistently detected", () => {});
  it("multi-pathology: all expected pathologies found", () => {});
});
```

10.5 Vitest Configuration

```
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    globals: true,
    include: ["tests/**/*.test.ts"],
    coverage: {
      provider: "v8",
      include: ["src/**/*.ts"],
      exclude: ["src/cli/**"],
      thresholds: {
        lines: 85,
        branches: 80,
        functions: 85,
        statements: 85,
      },
      reporter: ["text", "html", "lcov"],
    },
    testTimeout: 10000,
    setupFiles: ["tests/setup.ts"],
  },
});
```

11. CI/CD

11.1 CI (`.github/workflows/ci.yml`)

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with: { version: 9 }
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm run lint
      - run: pnpm run format:check
      - run: pnpm run typecheck

  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with: { version: 9 }
      - uses: actions/setup-node@v4
        with: { node-version: ${{ matrix.node-version }}, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm run test:ci
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: coverage-node-${{ matrix.node-version }}
          path: coverage/

  regression:
    runs-on: ubuntu-latest
```

```
    needs: test
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with: { version: 9 }
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm run test:regression
```

11.2 Release (`.github/workflows/release.yml`)

```
name: Release
on:
  push:
    tags: ["v*"]

jobs:
  publish:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with: { version: 9 }
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: pnpm
          registry-url: "https://registry.npmjs.org"
      - run: pnpm install --frozen-lockfile
      - run: pnpm run build
      - run: pnpm run test:ci
      - run: pnpm publish --provenance --access public
        env:
          NODE_AUTH_TOKEN: ${{ secrets.NPM_TOKEN }}
```

No docs workflow in v0.1.0 (TypeDoc deferred).

---

12. Documentation

12.1 README.md

```
# 🩺 AgentDoctor

**Diagnose your AI agents before they silently fail.**

AgentDoctor analyzes AI agent execution logs, detects 6 known failure
pathologies, and generates a health report with actionable fixes.

<!-- Badges: npm version, CI, coverage, Node 20+, Apache-2.0, TypeScript -->

## Quick Start

```bash
npx agentdoctor check my-agent-logs.json
```
```

Or pipe from stdin:

```
cat logs/session.json | npx agentdoctor check --stdin
```

What It Detects

Pathology	What Goes Wrong
Context Erosion	Context fills with noise; instructions get drowned out
Tool Thrashing	Agent calls tools in loops without converging
Instruction Drift	System prompt and tool schemas contradict each other
Recovery Blindness	Tool failures ignored — no retry, no fallback
Hallucinated Success	Agent claims tool succeeded when it failed
Silent Degradation	Gradual quality decline nobody notices

> Goal Hijacking detection coming in v0.2.0.

TypeScript API

```
import { analyze } from "agentdoctor";

const { diagnostics, healthScore, report } = await analyze({
  logFiles: ["logs/my-agent.json"],
```

```
});

console.log(`Health: ${healthScore.overallScore}/100 (${healthScore.overallGrade}
```

Supported Formats

LangChain (JS/Python) · OpenAI API · Generic JSON

> CrewAI and Vercel AI SDK parsers welcome — see CONTRIBUTING.md

Configuration

```
npx agentdoctor check logs/ --config my-thresholds.json
```

All defaults documented in docs/configuration.md.

Documentation

- Getting Started · Pathologies · Scoring · CLI · Config

Contributing

See CONTRIBUTING.md. We welcome:

- **New parsers** (CrewAI, Vercel AI SDK, Anthropic, custom)
- **Detection patterns** from real-world agent failures
- **Threshold calibration** from production data

About

Built by MLDeep Systems. AgentDoctor started as a consulting methodology for diagnosing AI agent failures. This open-source version provides the diagnostic framework.

For continuous monitoring, auto-remediation, and enterprise support → mldeep.systems/agentdoctor

License

Apache 2.0

12.2 CONTRIBUTING.md Must Cover

- Dev setup: `pnpm install`, `pnpm test`, `pnpm lint`
- Adding a new detector: implement `BaseDetector`, register in `detectors/index.t
- Adding a new parser: implement `BaseParser`, register in `parsers/index.ts`, ad
- **Wanted parsers:** CrewAI, Vercel AI SDK, Anthropic Claude, AWS Bedrock (good
- Running fixture generator: `pnpm run generate-fixtures`
- PR process: must pass CI, must include tests, must not reduce coverage

12.3 Other Files

- **SECURITY.md:** Responsible disclosure. AgentDoctor runs locally, no network.
- **CODE_OF_CONDUCT.md:** Contributor Covenant v2.1.
- **CHANGELOG.md:** Keep a Changelog. Start with v0.1.0.

---

13. Packaging & Distribution

13.1 `package.json`

```json
{
  "name": "agentdoctor",
  "version": "0.1.0",
  "description": "Diagnose your AI agents before they silently fail.",
  "type": "module",
  "main": "./dist/index.cjs",
  "module": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "exports": {
    ".": {
      "import": "./dist/index.js",
      "require": "./dist/index.cjs",
      "types": "./dist/index.d.ts"
    }
  },
  "bin": {
    "agentdoctor": "./dist/cli/index.js"
  },
  "files": ["dist", "README.md", "LICENSE", "CHANGELOG.md"],
  "scripts": {
    "build": "tsup",
    "dev": "tsup --watch",
    "lint": "eslint src/ tests/",
```

```
    "lint:fix": "eslint src/ tests/ --fix",
    "format": "prettier --write .",
    "format:check": "prettier --check .",
    "typecheck": "tsc --noEmit",
    "test": "vitest",
    "test:ci": "vitest run --coverage",
    "test:regression": "vitest run tests/regression/",
    "generate-fixtures": "tsx scripts/generate-fixtures.ts",
    "prepublishOnly": "pnpm run build"
  },
  "keywords": [
    "ai", "agents", "reliability", "diagnostics", "llm",
    "observability", "langchain", "openai", "health-check"
  ],
  "author": "Anmol <anmol@mldeep.systems>",
  "license": "Apache-2.0",
  "repository": {
    "type": "git",
    "url": "https://github.com/mldeep-systems/agentdoctor"
  },
  "homepage": "https://github.com/mldeep-systems/agentdoctor",
  "bugs": "https://github.com/mldeep-systems/agentdoctor/issues",
  "engines": { "node": ">=20" },
  "dependencies": {
    "commander": "^12.0.0",
    "picocolors": "^1.1.0",
    "zod": "^3.23.0"
  },
  "devDependencies": {
    "@typescript-eslint/eslint-plugin": "^7.0.0",
    "@typescript-eslint/parser": "^7.0.0",
    "eslint": "^9.0.0",
    "prettier": "^3.2.0",
    "tsup": "^8.0.0",
    "tsx": "^4.0.0",
    "typescript": "^5.4.0",
    "vitest": "^2.0.0",
    "@vitest/coverage-v8": "^2.0.0"
  }
}
```

3 production dependencies. That's it. `commander` (CLI), `picocolors` (colors), `zod` (config validation).

13.2 `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "lib": ["ES2022"],
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "isolatedModules": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist", "tests", "scripts"]
}
```

13.3 `tsup.config.ts`

```
import { defineConfig } from "tsup";

export default defineConfig({
  entry: {
    index: "src/index.ts",
    "cli/index": "src/cli/index.ts",
  },
  format: ["esm", "cjs"],
  dts: true,
  sourcemap: true,
  clean: true,
  target: "node20",
  splitting: false,
  shims: true,
});
```

14. GitHub Repository Setup

14.1 Branch Protection (`main`)

- Require CI passing (quality + test + regression)
- No direct pushes (require PR)

14.2 Issue Templates

Bug Report: Version, Node version, log format, description, expected, steps, log excerpt.

Pathology Report: Name, description, detection signal, example, severity.

14.3 Labels

`pathology` , `parser` , `scoring` , `report` , `cli` , `docs` , `good first issue` , `false-positive` , `false-negative` , `help wanted`

14.4 Topics

`ai-agents` , `reliability` , `diagnostics` , `llm` , `observability` , `langchain` , `openai` , `typescript` , `cli` , `health-check`

---

15. Acceptance Criteria — Ship Checklist

Code Quality

- `pnpm run lint` passes with zero errors
- `pnpm run format:check` passes
- `pnpm run typecheck` passes (strict mode, zero errors)
- No `any` types in public API
- All public exports have JSDoc comments
- ESM + CJS dual build works (`pnpm run build`)

Testing

- `pnpm run test:ci` passes on Node 20 and 22

- Coverage ≥ 85% lines
- All 6 detectors: positive detection tests pass
- All 6 detectors: false-positive tests pass (healthy → zero warnings/critical)
- All 3 parsers: parsing tests pass
- Parser auto-detection tests pass
- Scoring engine tests pass
- Both report formats produce valid output
- CLI tests cover `check`, `score`, all flags
- Integration: file → report passes for each pathology
- Regression: zero false positives on all healthy fixtures
- Regression: all 6 pathologies consistently detected

Fixtures

- 8 LangChain fixtures (1 healthy + 6 pathologies + 1 multi)
- 2 OpenAI fixtures (healthy + context-erosion)
- 2 Generic fixtures (healthy + mixed)
- All fixtures generated by `scripts/generate-fixtures.ts`
- All fixtures valid JSON/JSONL and parseable

Documentation

- README.md with badges, quickstart, pathology table
- CONTRIBUTING.md with dev setup, adding parsers/detectors, wanted list
- CHANGELOG.md with v0.1.0 entry
- CODE_OF_CONDUCT.md, SECURITY.md, LICENSE (Apache 2.0)
- All 6 pathology docs in docs/pathologies/
- docs/scoring.md, docs/cli.md, docs/configuration.md

Packaging

- `pnpm install` works from clean clone

- `pnpm run build` produces dist/
- `npx agentdoctor --version` outputs 0.1.0
- `npx agentdoctor check examples/sample-logs/healthy-agent.json` produces report
- `echo '...' | npx agentdoctor check --stdin` works
- `npm pack` produces valid tarball

GitHub

- `.gitignore` covers node_modules, dist, coverage, IDE files
- CI passes on fresh clone
- Issue templates render correctly
- Topics and description set
- Examples directory with working scripts

---

16. v0.2.0 Backlog

Features explicitly deferred from v0.1.0:

Feature	Why Deferred
Goal Hijacking detector	Imprecise heuristics, needs ACT dataset calibration
Cross-session silent degradation (rule 1)	Requires multi-session data users won't have yet
HTML report renderer	Engineering-heavy, low demand at launch
CrewAI parser	Unstable log format, community contribution opportunity
Vercel AI SDK parser	`experimental_telemetry` API, will change
`detect` / `validate` / `config` CLI commands	Convenience features, `check` handles implicitly
TypeDoc + GitHub Pages docs	Users run CLI, not library, in v0.1.0

Golden file tests	Brittle during early development
`--watch` mode	Nice to have, manual re-runs are fast enough
TOML config support	JSON only in Node.js ecosystem

Appendix A: Canonical Log Schemas

LangChain Callback Log

```
{
  "runs": [
    {
      "run_id": "uuid",
      "parent_run_id": "uuid | null",
      "type": "chain | llm | tool",
      "name": "string",
      "inputs": {},
      "outputs": {},
      "start_time": "ISO 8601",
      "end_time": "ISO 8601",
      "extra": {
        "metadata": {},
        "usage": { "prompt_tokens": 0, "completion_tokens": 0, "total_tokens": 0
      },
      "error": "string | null",
      "tags": [],
      "child_runs": []
    }
  ]
}
```

OpenAI API Log (JSONL)

```
{"request":{"model":"gpt-4","messages":[...],"tools":[...]},"response":{"choices"
```

Generic JSON Log (Minimum)

```
{
  "sessions": [
    {
```

```
      "session_id": "string",
      "messages": [
        { "role": "system|user|assistant|tool", "content": "string" }
      ],
      "tool_calls": [
        { "name": "string", "input": {}, "output": "string", "status": "success|e
      ]
    }
  ]
}
```

Appendix B: Health Score Formulas

Layer Score

```
layer_score = Σ(component_score_i × component_weight_i)
```

Each `component_score` : 0–100 via linear interpolation between healthy (100) and critical (0).

Composite Score

```
raw_score = Σ(layer_score_i × layer_weight_i)   // assessed layers only, weights
penalty = min(25, critical_count × 5) + min(10, warning_count × 2)
final_score = max(0, raw_score - penalty)
```

Grade Scale

Score	Grade	Meaning
90–100	A	Healthy
80–89	B	Good — minor issues
70–79	C	Concerning — address within 2 weeks
60–69	D	Degraded — address within 1 week
0–59	F	Critical — immediate attention

Appendix C: Sample Report Output

```
# 🩺 Agent Health Report

**Generated:** 2026-02-22T14:30:00Z
**Tool:** AgentDoctor v0.1.0
**Files:** 1 | **Sessions:** 1 | **Turns:** 20 | **Tool Calls:** 34

---

## Health Score: 58/100 (F)

| Layer | Score | Grade |
|-------|-------|-------|
| Context Health | 35/100 | F |
| Tool Reliability | 62/100 | D |
| Instruction Coherence | 88/100 | B |
| Recovery Robustness | — | *Requires continuous monitoring* |
| Output Quality | — | *Requires longitudinal data* |

---

## Findings

### 🔴 CRITICAL: Context growing unchecked — instruction share at 1.8%

**Pathology:** Context Erosion
**Confidence:** 0.94

The context window grew from 3,200 to 50,400 tokens without management.
Instructions now represent 1.8% of context, down from 17.2%.

<details><summary>Evidence</summary>

- Turn 1: 3,200 tokens (instruction share: 17.2%)
- Turn 10: 26,300 tokens (instruction share: 4.1%)
- Turn 20: 50,400 tokens (instruction share: 1.8%)
- Growth rate: ~2,360 tokens/turn

</details>

**Recommendation:** Implement context management: summarize tool outputs
older than 5 turns, add TTL-based memory pruning, or use a sliding window.

---
```

```
## What This Analysis Could Not Assess

- **Recovery Robustness** — requires simulated tool failures
- **Output Quality Baselines** — requires multiple sessions over time
- **Goal Hijacking** — coming in v0.2.0

For continuous monitoring with all 5 diagnostic layers, auto-remediation,
and real-time alerting → [mldeep.systems/agentdoctor](https://mldeep.systems/agen

---

## Recommendations

1. **[HIGH]** Implement context window management
2. **[MEDIUM]** Review search tool descriptions for ambiguity

---

*Generated by [AgentDoctor](https://github.com/mldeep-systems/agentdoctor) v0.1.0
```

END OF SPECIFICATION

Build it section by section. Validate against the checklist in Section 15. Ship it.